🚀 DAY 15 — CAMELOT-FIRST TABLE PIPELINE + SEMANTIC TABLE CHUNKING

✅ GOAL

Move from:

pdfplumber row chunks ❌

to:

Camelot table-semantic chunks ✅

for much better engineering retrieval.

✅ WHY DAY 15

Your current issue:

row-level chunking destroys engineering meaning ❌

Example:

Front pad Average

Front pad Maximum

Rear liner Average

become isolated embeddings.

So retrieval becomes weak.

✅ DAY 15 FIXES

- ✅ Camelot-first extraction
- ✅ better table reconstruction
- ✅ table-level chunking
- ✅ engineering semantic preservation
- ✅ stronger retrieval
- ✅ better comparisons
- ✅ cleaner context

✅ NEW ARCHITECTURE

PyMuPDF

↓

Page Text ✅

Camelot

↓

Primary Table Extraction ✅

pdfplumber

↓

Fallback ✅

Semantic Table Chunking ✅

↓

ChromaDB ✅

↓

Qwen2.5 ✅

✅ **TEMPLATE STRUCTURE**

app/

├── parser/

│ ├── camelot_parser.py ✅

│ ├── pdfplumber_table.py ✅ fallback

│

├── embeddings/

│ ├── chunker.py ✅ MODIFY

│

├── ingestion/

│ ├── pipeline.py ✅ MODIFY

✅ **STEP 1 — INSTALL**

pip install camelot-py[cv]

Windows may also need:

pip install ghostscript

And install Ghostscript system package.

✅ **STEP 2 — CREATE CAMELOT PARSER**

📄 **app/parser/camelot_parser.py**

```
import camelot
import pandas as pd

def extract_tables_camelot(pdf_path):

  extracted_tables = []

  try:

    tables = camelot.read_pdf(

      pdf_path,

      pages="all",

      flavor="lattice"

    )
```

```
for idx, table in enumerate(tables):

    df = table.df

    # ✅ remove empty rows
    df = df.dropna(
        axis=0,
        how="all"
    )

    # ✅ remove empty cols
    df = df.dropna(
        axis=1,
        how="all"
    )

    extracted_tables.append({

        "table_id":
            f"camelot_table_{idx}",

        "page":
            table.page,

        "df": df
    })

    print(
```

```
            f"✅ Camelot table extracted: {idx}"
        )

    except Exception as e:

        print(
            f"❌ Camelot extraction failed: {e}"
        )

    return extracted_tables
```

✅ **STEP 3 — UPDATE PIPELINE**

📄 **pipeline.py**

Add import:

```
from app.parser.camelot_parser import (
    extract_tables_camelot
)
```

Replace:

```
raw_tables = extract_tables_pdf(
    file_path
)
```

WITH:

```
# ✅ primary extraction
raw_tables = extract_tables_camelot(
    file_path
)
```

✅ fallback

if not raw_tables:

  raw_tables = extract_tables_pdf(

    file_path

)

✅ STEP 4 — SEMANTIC TABLE CHUNKING

📄 **app/embeddings/chunker.py**

Replace entire file.

---

def dataframe_to_chunks(table):

  df = table["df"]

  page = table["page"]

  table_id = table["table_id"]

  rows = []

  for _, row in df.iterrows():

    values = [

      str(v).strip()

      for v in row.values

```
        ]

        row_text = " | ".join(values)

        rows.append(row_text)

    # ✅ FULL TABLE CHUNK
    full_text = "\n".join(rows)

    return [{

        "id": table_id,

        "text": full_text,

        "metadata": {

            "page": page,

            "table_id": table_id,

            "source": "table"
        }
    }]
```

✅ WHY THIS CHUNKING IS BETTER

Instead of:

one row = one chunk ❌

you now get:

whole engineering section = one chunk ✅

✅ **EXAMPLE NEW CHUNK**

P201 BRAKE TEMPERATURE MAPPING

Front pad peak temperature

Average | 104 | 104 | 82

Maximum | 163 | 179 | 188

Rear liner peak temperature

Average | 94 | 98 | 72

Maximum | 153 | 153 | 146

✅ semantically complete
✅ engineering relationships preserved
✅ much better embeddings

✅ **STEP 5 — DELETE OLD DB**

IMPORTANT.

Old embeddings are row-based.

Delete:

chroma_db/

engineering.db

✅ **STEP 6 — REINGEST**

Run:

python main.py

First run will:

✅ Camelot extraction

✅ semantic table chunking

✅ fresh embeddings

✅ fresh DuckDB

✅ EXPECTED IMPROVEMENTS

Now queries like:

front pad average temperature

compare rear liner

GHD values

maximum temperatures

will work MUCH better.

✅ WHY THIS IS THE CORRECT ARCHITECTURE

Engineering PDFs are:

table-semantic ✅

NOT:

sentence-semantic ❌

Meaning exists across:

- neighboring rows
- Average/Maximum pairs
- CDT/GHD/HSD relationships
- grouped metrics

So:

table chunking > row chunking

✅ CURRENT SYSTEM

PyMuPDF ✅

Camelot ✅

pdfplumber fallback ✅

Semantic Table Chunking ✅

bge embeddings ✅

ChromaDB ✅

DuckDB ✅

Qwen2.5 ✅

✅ **YOU ARE NOW HERE**

✅ DAY 15 COMPLETE

Next:

DAY 16

→ hybrid table + text retrieval

→ adaptive chunk sizing

→ section-aware retrieval

→ multi-document reasoning

→ engineering dashboard